

Technical Specifications

Real-Time AR Soil/Crop Analysis System

Babylon.js & AI Model Implementation

1. Babylon.js AR Scene Setup

1.1 Core Scene Initialization

```
typescript
```

```
// BabylonARScene.ts
import * as BABYLON from '@babylonjs/core';
import '@babylonjs/loaders';
import { WebXRFeatureName, WebXRPlaneDetector } from '@babylonjs/core/XR';

export class BabylonARScene {
  private engine: BABYLON.Engine;
  private scene: BABYLON.Scene;
  private xrHelper: BABYLON.WebXRDefaultExperience;
  private planeDetector: WebXRPlaneDetector;
  private hitTest: BABYLON.WebXRHitTest;

  constructor(canvas: HTMLCanvasElement) {
    // Initialize Babylon.js engine
    this.engine = new BABYLON.Engine(canvas, true, {
      preserveDrawingBuffer: true,
      stencil: true,
      antialias: true,
      powerPreference: "high-performance"
    });

    // Create scene
    this.scene = new BABYLON.Scene(this.engine);
    this.setupScene();
  }

  private async setupScene(): Promise<void> {
    // Configure scene for AR
    this.scene.clearColor = new BABYLON.Color4(0, 0, 0, 0);

    // Create AR camera
    const camera = new BABYLON.FreeCamera(
      "arCamera",
      BABYLON.Vector3.Zero(),
      this.scene
    );

    // Minimal lighting for AR
    const light = new BABYLON.HemisphericLight(
      "light",
      new BABYLON.Vector3(0, 1, 0),
      this.scene
    );
  }
}
```

```
light.intensity = 0.7;

// Initialize WebXR
await this.initializeXR();

// Start render loop
this.engine.runRenderLoop(() => {
  this.scene.render();
});
}

private async initializeXR(): Promise<void> {
  try {
    // Create WebXR experience
    this.xrHelper = await this.scene.createDefaultXRExperienceAsync({
      uiOptions: {
        sessionMode: "immersive-ar",
        referenceSpaceType: "local-floor",
        optionalFeatures: [
          "plane-detection",
          "hit-test",
          "anchors",
          "light-estimation",
          "dom-overlay"
        ]
      },
      inputOptions: {
        doNotLoadControllerMeshes: true
      }
    });

    // Enable plane detection
    await this.setupPlaneDetection();

    // Enable hit testing
    await this.setupHitTesting();

    console.log("WebXR initialized successfully");
  } catch (error) {
    console.error("WebXR initialization failed:", error);
    // Fallback to non-AR mode
    this.initializeFallbackMode();
  }
}
```

```
}
```

```
}
```

1.2 Plane Detection System

typescript

```
// PlaneDetectionManager.ts
```

```
export class PlaneDetectionManager {  
  private planeDetector: WebXRPlaneDetector;  
  private detectedPlanes: Map<string, DetectedPlane> = new Map();  
  private groundPlanes: DetectedPlane[] = [];  
  
  constructor(  
    private xrHelper: BABYLON.WebXRDefaultExperience,  
    private scene: BABYLON.Scene  
  ) {}  
  
  async initialize(): Promise<void> {  
    const featuresManager = this.xrHelper.baseExperience.featuresManager;  
  
    // Enable plane detection feature  
    this.planeDetector = featuresManager.enableFeature(  
      BABYLON.WebXRFeatureName.PLANE_DETECTION,  
      "latest",  
      {  
        preferredDetectorOptions: {  
          preferredPolygonType: "polygon",  
          updateRate: 10 // 10Hz update rate  
        }  
      }  
    ) as WebXRPlaneDetector;  
  
    // Subscribe to plane updates  
    this.planeDetector.onPlaneAddedObservable.add(this.onPlaneAdded.bind(this));  
    this.planeDetector.onPlaneUpdatedObservable.add(this.onPlaneUpdated.bind(this));  
    this.planeDetector.onPlaneRemovedObservable.add(this.onPlaneRemoved.bind(this));  
  }  
  
  private onPlaneAdded(plane: BABYLON.IWebXRPlane): void {  
    // Create visual representation  
    const planeMesh = this.createPlaneMesh(plane);  
  
    // Classify plane (ground vs wall vs ceiling)  
    const classification = this.classifyPlane(plane);  
  
    const detectedPlane: DetectedPlane = {  
      id: plane.id,  
      plane: plane,  
      mesh: planeMesh,  
    };  
  }  
}
```

```

    classification: classification,
    lastUpdate: Date.now()
};

this.detectedPlanes.set(plane.id, detectedPlane);

// Track ground planes specifically
if (classification === PlaneType.GROUND) {
    this.groundPlanes.push(detectedPlane);
    this.onGroundDetected(detectedPlane);
}
}

private classifyPlane(plane: BABYLON.IWebXRPlane): PlaneType {
    // Get plane normal vector
    const normal = plane.transformationMatrix.getRow(1);

    // Calculate angle from up vector
    const upVector = new BABYLON.Vector3(0, 1, 0);
    const angle = Math.acos(
        BABYLON.Vector3.Dot(normal.toVector3(), upVector)
    ) * (180 / Math.PI);

    // Classification based on normal angle
    if (angle < 30) {
        return PlaneType.GROUND; // Horizontal plane
    } else if (angle > 150) {
        return PlaneType.CEILING; // Inverted horizontal
    } else {
        return PlaneType.WALL; // Vertical plane
    }
}

private createPlaneMesh(plane: BABYLON.IWebXRPlane): BABYLON.Mesh {
    // Create polygon mesh from plane vertices
    const positions: number[] = [];
    const indices: number[] = [];

    // Convert plane polygon to mesh vertices
    if (plane.polygon) {
        plane.polygon.forEach((point, index) => {
            positions.push(point.x, 0, point.z);
        });
    }
}

```

```

    // Create triangulation
    for (let i = 1; i < plane.polygon.length - 1; i++) {
        indices.push(0, i, i + 1);
    }
}

// Create custom mesh
const customMesh = new BABYLON.Mesh("plane_" + plane.id, this.scene);
const vertexData = new BABYLON.VertexData();
vertexData.positions = positions;
vertexData.indices = indices;
vertexData.applyToMesh(customMesh);

// Apply semi-transparent material for ground planes
const material = new BABYLON.StandardMaterial("planeMat", this.scene);
material.diffuseColor = new BABYLON.Color3(0, 1, 0);
material.alpha = 0.3;
material.backFaceCulling = false;
customMesh.material = material;

// Apply transformation
customMesh.setPreTransformMatrix(plane.transformationMatrix);

return customMesh;
}

private onGroundDetected(plane: DetectedPlane): void {
    console.log("Ground plane detected:", plane.id);
    // Trigger ground analysis
    this.analyzeGroundArea(plane);
}

private async analyzeGroundArea(plane: DetectedPlane): Promise<void> {
    // Calculate plane area
    const area = this.calculatePlaneArea(plane.polygon);

    // Get center point for analysis
    const centerPoint = this.getPlaneCenter(plane.polygon);

    // Trigger AI analysis at this location
    await AIAnalyzer.analyzeLocation(centerPoint, area);
}
}

```

```
enum PlaneType {  
    GROUND = "ground",  
    WALL = "wall",  
    CEILING = "ceiling",  
    UNKNOWN = "unknown"  
}  
  
interface DetectedPlane {  
    id: string;  
    plane: BABYLON.IWebXRPlane;  
    mesh: BABYLON.Mesh;  
    classification: PlaneType;  
    lastUpdate: number;  
}
```

2. Hit Testing Implementation

2.1 WebXR Hit Test Manager

typescript


```
// HitTestManager.ts
```

```
export class HitTestManager {
  private hitTestSource: BABYLON.IWebXRHitTestSource;
  private currentHitResult: BABYLON.IWebXRHitResult | null = null;
  private hitMarker: BABYLON.Mesh;
  private isActive: boolean = false;

  constructor(
    private xrHelper: BABYLON.WebXRDefaultExperience,
    private scene: BABYLON.Scene
  ) {
    this.createHitMarker();
  }

  async initialize(): Promise<void> {
    const featuresManager = this.xrHelper.baseExperience.featuresManager;

    // Enable hit testing
    const hitTest = featuresManager.enableFeature(
      BABYLON.WebXRFeatureName.HIT_TEST,
      "latest",
      {
        entityTypes: ["plane", "mesh"],
        offsetRay: { origin: { x: 0, y: 0, z: 0 }, direction: { x: 0, y: 0, z: -1 } },
        useReferenceSpace: true
      }
    ) as BABYLON.WebXRHitTest;

    // Subscribe to hit test results
    hitTest.onHitTestResultObservable.add((results) => {
      if (results.length > 0) {
        this.processHitResult(results[0]);
      } else {
        this.hideHitMarker();
      }
    });
  }

  private createHitMarker(): void {
    // Create reticle/marker for hit point
    this.hitMarker = BABYLON.MeshBuilder.CreateTorus(
      "hitMarker",
      {

```

```

        diameter: 0.15,
        thickness: 0.01,
        tessellation: 32
    },
    this.scene
);

const material = new BABYLON.StandardMaterial("hitMarkerMat", this.scene);
material.diffuseColor = new BABYLON.Color3(1, 1, 0);
material.emissiveColor = new BABYLON.Color3(1, 1, 0);
this.hitMarker.material = material;
this.hitMarker.isVisible = false;
}

private processHitResult(result: BABYLON.IWebXRHitResult): void {
    this.currentHitResult = result;

    // Update marker position
    const matrix = result.transformationMatrix;
    this.hitMarker.setPreTransformMatrix(matrix);
    this.hitMarker.isVisible = true;

    // Check if hitting ground plane
    if (this.isGroundHit(result)) {
        this.onGroundHit(result);
    }
}

private isGroundHit(result: BABYLON.IWebXRHitResult): boolean {
    // Extract normal from transformation matrix
    const normal = result.transformationMatrix.getRow(1).toVector3();

    // Check if normal points upward (ground plane)
    const dotProduct = BABYLON.Vector3.Dot(
        normal,
        BABYLON.Vector3.Up()
    );

    return dotProduct > 0.8; // Within 36 degrees of up
}

private onGroundHit(result: BABYLON.IWebXRHitResult): void {
    // Convert hit point to world coordinates
    const worldPos = this.getWorldPosition(result);

```

```

// Trigger analysis at this point
this.analyzePoint(worldPos);
}

private getWorldPosition(result: BABYLON.IWebXRHitResult): BABYLON.Vector3 {
    const matrix = result.transformationMatrix;
    return new BABYLON.Vector3(
        matrix.m[12],
        matrix.m[13],
        matrix.m[14]
    );
}

async analyzePoint(worldPos: BABYLON.Vector3): Promise<AnalysisResult> {
    // Get device GPS location
    const deviceGPS = await this.getCurrentGPS();

    // Calculate GPS offset from hit point
    const hitGPS = this.worldPosToGPS(worldPos, deviceGPS);

    // Trigger full analysis pipeline
    const result = await this.runAnalysisPipeline(hitGPS, worldPos);

    return result;
}

private worldPosToGPS(
    worldPos: BABYLON.Vector3,
    deviceGPS: GPSCoordinate
): GPSCoordinate {
    // Calculate offset in meters
    const offsetX = worldPos.x; // East-West
    const offsetZ = worldPos.z; // North-South

    // Convert to GPS offset (approximate)
    const latOffset = offsetZ / 111111; // meters to degrees
    const lonOffset = offsetX / (111111 * Math.cos(deviceGPS.lat * Math.PI / 180));

    return {
        lat: deviceGPS.lat + latOffset,
        lon: deviceGPS.lon + lonOffset
    };
}

```

```
}
```

```
}
```

3. AI Model System

3.1 TensorFlow.js Model Manager

```
typescript
```

```

// AIModelManager.ts
import * as tf from '@tensorflow/tfjs';

export class AIModelManager {
  private models: Map<ModelType, tf.LayersModel> = new Map();
  private isReady: boolean = false;

  // Model configurations
  private readonly MODEL_CONFIGS = {
    terrain: {
      url: '/models/terrain-classifier/model.json',
      inputShape: [224, 224, 3],
      outputClasses: ['soil', 'grass', 'crop', 'concrete', 'water', 'unknown'],
      preprocessing: 'imagenet'
    },
    crop: {
      url: '/models/crop-identifier/model.json',
      inputShape: [224, 224, 3],
      outputClasses: ['corn', 'wheat', 'soybean', 'pasture', 'bare', 'unknown'],
      preprocessing: 'custom'
    },
    health: {
      url: '/models/health-analyzer/model.json',
      inputShape: [224, 224, 3],
      outputType: 'regression', // 0-100 score
      preprocessing: 'normalize'
    }
  };

  async initialize(): Promise<void> {
    try {
      // Set TensorFlow.js backend
      await tf.setBackend('webgl');

      // Enable WebGL optimizations
      tf.env().set('WEBGL_FORCE_F16_TEXTURES', false);
      tf.env().set('WEBGL_PACK', true);
      tf.env().set('WEBGL_CONV_IM2COL', true);

      // Load all models in parallel
      const loadPromises = Object.entries(this.MODEL_CONFIGS).map(
        async ([type, config]) => {
          const model = await this.loadModel(config.url);

```

```

    this.models.set(type as ModelType, model);

    // Warm up model with dummy input
    await this.warmupModel(model, config.inputShape);
  }
};

await Promise.all(loadPromises);
this.isReady = true;

console.log("All AI models loaded successfully");
} catch (error) {
  console.error("Failed to load AI models:", error);
  throw error;
}
}

private async loadModel(url: string): Promise<tf.LayersModel> {
  try {
    const model = await tf.loadLayersModel(url);

    // Compile model for inference
    model.compile({
      optimizer: tf.train.adam(),
      loss: 'categoricalCrossentropy',
      metrics: ['accuracy']
    });

    return model;
  } catch (error) {
    console.error(`Failed to load model from ${url}:`, error);
    throw error;
  }
}

private async warmupModel(
  model: tf.LayersModel,
  inputShape: number[]
): Promise<void> {
  // Create dummy input
  const dummyInput = tf.zeros([1, ...inputShape]);

  // Run inference to warm up GPU
  const prediction = model.predict(dummyInput) as tf.Tensor;

```

```

// Clean up
dummyInput.dispose();
prediction.dispose();
}

async analyzeFrame(
  videoFrame: HTMLVideoElement | ImageData
): Promise<AnalysisResult> {
  if (!this.isReady) {
    throw new Error("Models not ready");
  }

  // Extract and preprocess frame
  const tensor = await this.preprocessFrame(videoFrame);

  try {
    // Run all models in parallel
    const [terrainResult, cropResult, healthResult] = await Promise.all([
      this.classifyTerrain(tensor),
      this.identifyCrop(tensor),
      this.analyzeHealth(tensor)
    ]);

    // Format result to match Pixel Hunt structure
    return this.formatAnalysisResult(
      terrainResult,
      cropResult,
      healthResult
    );
  } finally {
    // Clean up tensors
    tensor.dispose();
  }
}

private async preprocessFrame(
  input: HTMLVideoElement | ImageData
): Promise<tf.Tensor4D> {
  // Convert to tensor
  let tensor = tf.browser.fromPixels(input);

  // Resize to model input size
  tensor = tf.image.resizeBilinear(tensor, [224, 224]);

```

```

// Normalize to [0, 1]
tensor = tensor.div(255.0);

// Add batch dimension
return tensor.expandDims(0) as tf.Tensor4D;
}

private async classifyTerrain(input: tf.Tensor4D): Promise<TerrainResult> {
  const model = this.models.get(ModelType.TERRAIN);
  if (!model) throw new Error("Terrain model not loaded");

  // Run inference
  const prediction = model.predict(input) as tf.Tensor;
  const probabilities = await prediction.array() as number[][];

  // Get top class
  const classes = this.MODEL_CONFIGS.terrain.outputClasses;
  const topIndex = probabilities[0].indexOf(Math.max(...probabilities[0]));

  prediction.dispose();

  return {
    type: classes[topIndex],
    confidence: probabilities[0][topIndex],
    allProbabilities: Object.fromEntries(
      classes.map((cls, i) => [cls, probabilities[0][i]])
    )
  };
}

private async identifyCrop(input: tf.Tensor4D): Promise<CropResult> {
  const model = this.models.get(ModelType.CROP);
  if (!model) throw new Error("Crop model not loaded");

  const prediction = model.predict(input) as tf.Tensor;
  const probabilities = await prediction.array() as number[][];

  const classes = this.MODEL_CONFIGS.crop.outputClasses;
  const topIndex = probabilities[0].indexOf(Math.max(...probabilities[0]));

  prediction.dispose();

  return {

```



```
        crop: classes[topIndex],
        confidence: probabilities[0][topIndex]
    };
}
```

```
private async analyzeHealth(input: tf.Tensor4D): Promise<HealthResult> {
    const model = this.models.get(ModelType.HEALTH);
    if (!model) throw new Error("Health model not loaded");

    const prediction = model.predict(input) as tf.Tensor;
    const score = await prediction.array() as number[][];

    prediction.dispose();

    // Convert to percentage
    const healthScore = Math.round(score[0][0] * 100);

    return {
        score: healthScore,
        status: this.getHealthStatus(healthScore)
    };
}
```

```
private getHealthStatus(score: number): string {
    if (score >= 90) return "Excellent";
    if (score >= 75) return "Good";
    if (score >= 60) return "Fair";
    if (score >= 40) return "Poor";
    return "Critical";
}
```

```
private formatAnalysisResult(
    terrain: TerrainResult,
    crop: CropResult,
    health: HealthResult
): AnalysisResult {
    return {
        type: terrain.type === 'crop' ? crop.crop : terrain.type,
        health: health.score,
        confidence: (terrain.confidence + crop.confidence) / 2,
        details: {
            terrain: terrain.type,
            crop: crop.crop,
            healthStatus: health.status
        }
    };
}
```

```
    }  
    };  
}  
  
enum ModelType {  
    TERRAIN = "terrain",  
    CROP = "crop",  
    HEALTH = "health"  
}
```

4. Data Integration Pipeline

4.1 Real-time Data Processor

```
typescript
```

```
// DataProcessor.ts
export class DataProcessor {
  private nasaClient: NASADataClient;
  private aiManager: AIModelManager;
  private cache: DataCache;
  private updateInterval: number = 100; // 10Hz update rate
  private lastUpdate: number = 0;

  constructor() {
    this.nasaClient = new NASADataClient();
    this.aiManager = new AIModelManager();
    this.cache = new DataCache();
  }

  async processARPoint(
    hitResult: BABYLON.IWebXRHitResult,
    cameraFrame: HTMLVideoElement
  ): Promise<PixelHuntData> {
    // Throttle updates
    const now = Date.now();
    if (now - this.lastUpdate < this.updateInterval) {
      return this.cache.getLastResult();
    }
    this.lastUpdate = now;

    // Extract position
    const worldPos = this.extractWorldPosition(hitResult);
    const gpsCoord = await this.calculateGPSCoordinate(worldPos);

    // Run parallel analysis
    const [aiResult, nasaData] = await Promise.all([
      this.aiManager.analyzeFrame(cameraFrame),
      this.nasaClient.fetchData(gpsCoord)
    ]);

    // Format to Pixel Hunt structure
    const result = this.formatToPixelHunt(
      gpsCoord,
      aiResult,
      nasaData
    );

    // Cache result
```

```

    this.cache.store(result);

    return result;
}

private formatToPixelHunt(
    gps: GPSCoordinate,
    ai: AnalysisResult,
    nasa: NASADData
): PixelHuntData {
    // Convert GPS to pixel grid position (matching original Pixel Hunt)
    const pixelPos = this.gpsToPixelGrid(gps);

    return {
        pixel: [pixelPos.x, pixelPos.y],
        moisture: nasa.soilMoisture || 0,
        ndvi: nasa.ndvi || 0,
        temperature: nasa.surfaceTemp || 0,
        type: ai.type || "unknown",
        health: ai.health || 0,
        index: this.calculateIndex(pixelPos)
    };
}

private gpsToPixelGrid(gps: GPSCoordinate): PixelPosition {
    // Map GPS to grid system (same as original Pixel Hunt)
    // Assuming 0.01 degree grid cells
    const gridSize = 0.01;

    return {
        x: Math.floor(gps.lon / gridSize),
        y: Math.floor(gps.lat / gridSize)
    };
}

private calculateIndex(pixel: PixelPosition): number {
    // Calculate unique index for grid position
    // Same formula as original Pixel Hunt
    const gridWidth = 360 / 0.01; // Total grid width
    return pixel.y * gridWidth + pixel.x;
}
}

```

4.2 NASA Data Client

typescript

```

// NASADDataClient.ts
export class NASADDataClient {
  private readonly API_BASE = 'https://api.nasa.gov';
  private cache: Map<string, CachedData> = new Map();
  private readonly CACHE_TTL = 300000; // 5 minutes

  async fetchData(gps: GPSCoordinate): Promise<NASADData> {
    // Check cache first
    const cacheKey = this.getCacheKey(gps);
    const cached = this.cache.get(cacheKey);

    if (cached && Date.now() - cached.timestamp < this.CACHE_TTL) {
      return cached.data;
    }

    // Fetch fresh data
    const [moisture, ndvi, temperature] = await Promise.all([
      this.fetchSoilMoisture(gps),
      this.fetchNDVI(gps),
      this.fetchTemperature(gps)
    ]);

    const data: NASADData = {
      soilMoisture: moisture,
      ndvi: ndvi,
      surfaceTemp: temperature,
      timestamp: Date.now()
    };

    // Cache result
    this.cache.set(cacheKey, {
      data: data,
      timestamp: Date.now()
    });

    return data;
  }

  private async fetchSoilMoisture(gps: GPSCoordinate): Promise<number> {
    // SMAP API call
    const url = `${this.API_BASE}/smap/soil_moisture`;
    const params = new URLSearchParams({
      lat: gps.lat.toString(),

```

```
lon: gps.lon.toString(),
api_key: process.env.NASA_API_KEY
});

try {
  const response = await fetch(`${url}?${params}`);
  const data = await response.json();
  return data.moisture_percent || 0;
} catch (error) {
  console.error("Failed to fetch soil moisture:", error);
  return 0;
}
}

private getCacheKey(gps: GPSCoordinate): string {
  // Round to nearest 0.01 degree for caching
  const lat = Math.round(gps.lat * 100) / 100;
  const lon = Math.round(gps.lon * 100) / 100;
  return `${lat}_${lon}`;
}
}
```

5. AR Overlay Rendering

5.1 Data Overlay System

```
typescript
```

```
// AROverlay.ts
import * as BABYLON from '@babylonjs/core';
import * as GUI from '@babylonjs/gui';

export class AROverlay {
    private advancedTexture: GUI.AdvancedDynamicTexture;
    private dataPanel: GUI.Rectangle;
    private textBlock: GUI.TextBlock;
    private isVisible: boolean = false;

    constructor(private scene: BABYLON.Scene) {
        this.setupGUI();
    }

    private setupGUI(): void {
        // Create fullscreen UI
        this.advancedTexture = GUI.AdvancedDynamicTexture.CreateFullscreenUI(
            "AR_UI",
            true,
            this.scene
        );

        // Create data panel (matching Pixel Hunt style)
        this.dataPanel = new GUI.Rectangle("dataPanel");
        this.dataPanel.width = "200px";
        this.dataPanel.height = "180px";
        this.dataPanel.cornerRadius = 10;
        this.dataPanel.color = "#00FF00";
        this.dataPanel.thickness = 2;
        this.dataPanel.background = "rgba(0, 0, 0, 0.8)";

        // Create text block
        this.textBlock = new GUI.TextBlock("dataText");
        this.textBlock.color = "#00FF00";
        this.textBlock.fontSize = "14px";
        this.textBlock.fontFamily = "monospace";
        this.textBlock.textHorizontalAlignment = GUI.Control.HORIZONTAL_ALIGNMENT_LEFT;
        this.textBlock.textVerticalAlignment = GUI.Control.VERTICAL_ALIGNMENT_TOP;
        this.textBlock.paddingLeft = "10px";
        this.textBlock.paddingTop = "10px";

        this.dataPanel.addControl(this.textBlock);
        this.advancedTexture.addControl(this.dataPanel);
    }
}
```



```

// Initially hidden
this.hide();
}

updateData(data: PixelHuntData): void {
    // Format text exactly like Pixel Hunt
    const text = `Pixel [${data.pixel[0]}, ${data.pixel[1]}]
Moisture: ${data.moisture.toFixed(1)}%
NDVI: ${data.ndvi.toFixed(3)}
Temp: ${data.temperature.toFixed(1)}°C
Type: ${data.type}
Health: ${data.health}%
Index: ${data.index}`;

    this.textBlock.text = text;
    this.show();
}

positionAtHitPoint(hitResult: BABYLON.IWebXRHitResult): void {
    if (!hitResult) return;

    // Get world position from hit result
    const worldMatrix = hitResult.transformationMatrix;
    const worldPos = new BABYLON.Vector3(
        worldMatrix.m[12],
        worldMatrix.m[13] + 0.2, // Slightly above ground
        worldMatrix.m[14]
    );

    // Convert to screen coordinates
    const camera = this.scene.activeCamera;
    if (!camera) return;

    const engine = this.scene.getEngine();
    const viewport = camera.viewport.toGlobal(
        engine.getRenderWidth(),
        engine.getRenderHeight()
    );

    const screenPos = BABYLON.Vector3.Project(
        worldPos,
        BABYLON.Matrix.Identity(),
        this.scene.getTransformMatrix(),

```

```
        viewport
    );

    // Position panel at screen coordinates
    this.dataPanel.leftInPixels = screenPos.x - 100;
    this.dataPanel.topInPixels = screenPos.y - 90;
}

show(): void {
    this.dataPanel.isVisible = true;
    this.isVisible = true;
}

hide(): void {
    this.dataPanel.isVisible = false;
    this.isVisible = false;
}
}
```

6. Performance Optimization

6.1 Frame Processing Optimization

```
typescript
```

```
// FrameProcessor.ts
export class FrameProcessor {
  private processingQueue: FrameData[] = [];
  private isProcessing: boolean = false;
  private frameSkip: number = 2; // Process every 3rd frame
  private frameCount: number = 0;

  async processVideoStream(video: HTMLVideoElement): Promise<void> {
    // Skip frames for performance
    this.frameCount++;
    if (this.frameCount % this.frameSkip !== 0) {
      return;
    }

    // Don't queue if already processing
    if (this.isProcessing) {
      return;
    }

    this.isProcessing = true;

    try {
      // Extract frame efficiently
      const frame = await this.extractFrame(video);

      // Process with Web Workers for non-blocking
      const result = await this.processInWorker(frame);

      // Update UI
      this.updateDisplay(result);
    } finally {
      this.isProcessing = false;
    }
  }

  private async extractFrame(video: HTMLVideoElement): Promise<ImageData> {
    const canvas = document.createElement('canvas');
    canvas.width = 224; // Model input size
    canvas.height = 224;

    const ctx = canvas.getContext('2d');
    ctx.drawImage(video, 0, 0, 224, 224);
  }
}
```

```
    return ctx.getImageData(0, 0, 224, 224);
  }

  private async processInWorker(frame: ImageData): Promise<any> {
    return new Promise((resolve, reject) => {
      const worker = new Worker('/workers/ai-processor.js');

      worker.postMessage({ type: 'process', frame: frame });

      worker.onmessage = (event) => {
        resolve(event.data);
        worker.terminate();
      };

      worker.onerror = reject;
    });
  }
}
```

6.2 Memory Management

typescript

```
// MemoryManager.ts
export class MemoryManager {
  private readonly MAX_MEMORY_MB = 512;
  private memoryUsage: number = 0;

  constructor() {
    this.startMonitoring();
  }

  private startMonitoring(): void {
    setInterval(() => {
      if ('memory' in performance) {
        const memory = (performance as any).memory;
        const usedMB = memory.usedJSHeapSize / 1048576;

        if (usedMB > this.MAX_MEMORY_MB) {
          this.performCleanup();
        }
      }
    }, 5000);
  }

  private performCleanup(): void {
    // Clear old cache entries
    DataCache.getInstance().clearOldEntries();

    // Dispose unused tensors
    tf.disposeVariables();

    // Force garbage collection if available
    if ('gc' in window) {
      (window as any).gc();
    }

    console.log("Memory cleanup performed");
  }
}
```

7. Type Definitions

7.1 Core Types

typescript

```
// types.d.ts
```

```
// Pixel Hunt Data Format (unchanged)
```

```
interface PixelHuntData {  
  pixel: [number, number];  
  moisture: number;  
  ndvi: number;  
  temperature: number;  
  type: string;  
  health: number;  
  index: number;  
}
```

```
// GPS Coordinate
```

```
interface GPSCoordinate {  
  lat: number;  
  lon: number;  
  accuracy?: number;  
}
```

```
// Analysis Results
```

```
interface AnalysisResult {  
  type: string;  
  health: number;  
  confidence: number;  
  details: {  
    terrain: string;  
    crop: string;  
    healthStatus: string;  
  };  
}
```

```
// NASA Data
```

```
interface NASAData {  
  soilMoisture: number;  
  ndvi: number;  
  surfaceTemp: number;  
  timestamp: number;  
}
```

```
// Terrain Classification Result
```

```
interface TerrainResult {  
  type: string;
```

```
confidence: number;
allProbabilities: Record<string, number>;
}

// Crop Identification Result
interface CropResult {
  crop: string;
  confidence: number;
}

// Health Analysis Result
interface HealthResult {
  score: number;
  status: string;
}

// Cached Data
interface CachedData {
  data: any;
  timestamp: number;
}

// Pixel Position
interface PixelPosition {
  x: number;
  y: number;
}

// Frame Data
interface FrameData {
  imageData: ImageData;
  timestamp: number;
  gps: GPSCoordinate;
}
```

8. Configuration

8.1 Application Configuration

```
typescript
```



```
// config.ts
export const AppConfig = {
  // Babylon.js settings
  babylon: {
    engineOptions: {
      preserveDrawingBuffer: true,
      stencil: true,
      antialias: true,
      powerPreference: "high-performance"
    },
    xrOptions: {
      sessionMode: "immersive-ar",
      referenceSpaceType: "local-floor",
      requiredFeatures: ["plane-detection", "hit-test"],
      optionalFeatures: ["anchors", "light-estimation", "dom-overlay"]
    }
  },

  // AI Model settings
  ai: {
    modelPath: "/models/",
    warmupOnLoad: true,
    inferenceBackend: "webgl",
    maxConcurrentInferences: 2,
    frameSkip: 2 // Process every 3rd frame
  },

  // Data settings
  data: {
    updateRateHz: 10,
    cacheTimeMs: 300000, // 5 minutes
    coordinatePrecision: 6, // decimal places
    gridCellSize: 0.01 // degrees
  },

  // Performance settings
  performance: {
    targetFPS: 30,
    maxMemoryMB: 512,
    workerPoolSize: 2,
    textureResolution: 1024
  },
}
```

```
// NASA API settings
```

```
nasa: {  
  baseUrl: "https://api.nasa.gov",  
  endpoints: {  
    smap: "/smap/soil_moisture",  
    modis: "/modis/vegetation",  
    temperature: "/modis/surface_temp"  
  },  
  rateLimit: {  
    requestsPerMinute: 60,  
    burstSize: 10  
  }  
}  
};
```

Appendix A: Model Training Specifications

Training Data Requirements

- **Terrain Dataset:** 100,000 labeled images (soil, grass, crop, concrete, water)
- **Crop Dataset:** 75,000 labeled crop images (corn, wheat, soybean, etc.)
- **Health Dataset:** 50,000 images with health scores (0-100)

Model Architecture

- **Base:** MobileNetV3 (optimized for mobile inference)
- **Input Size:** 224x224x3
- **Quantization:** INT8 for reduced size
- **Size Targets:** <10MB per model

Performance Metrics

- **Inference Time:** <100ms on mobile GPU
- **Accuracy:** >85% for classification
- **Memory Usage:** <50MB total for all models